

Lab 8: Test-Driven Development with AI – Generating and Working with Test Cases

Name: N.Adithya

Roll No: 2503A52413

Batch No: 14

Task Description #1 (Password Strength Validator – Apply AI in Security Context)

Task:

Apply AI to generate at least 3 assert test cases for `is_strong_password(password)` and implement the validator function.

Input:

```
def is_strong_password(password):  
    # Check minimum length  
    if len(password) < 8:  
        return False  
  
    # Password must not contain spaces  
    if " " in password:  
        return False  
  
    # Check for uppercase, lowercase, digit, and special character  
    has_upper = False
```

```
has_lower = False
```

```
has_digit = False
```

```
has_special = False
```

```
for char in password:
```

```
    if char.isupper():
```

```
        has_upper = True
```

```
    elif char.islower():
```

```
        has_lower = True
```

```
    elif char.isdigit():
```

```
        has_digit = True
```

```
    else:
```

```
        has_special = True
```

```
return has_upper and has_lower and has_digit and has_special
```

```
# AI-generated assert test cases
```

```
assert is_strong_password("Abcd@123") == True
```

```
assert is_strong_password("abcd123") == False
```

```
assert is_strong_password("ABCD@1234") == False
```

```
assert is_strong_password("abcdef@12") == True
```

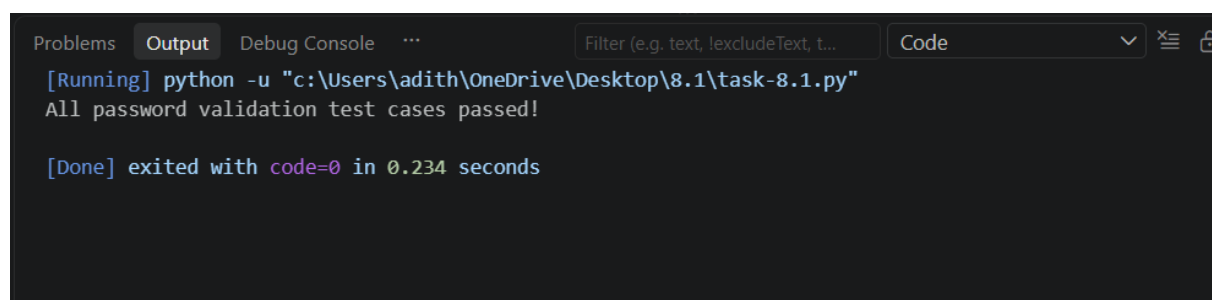
```
assert is_strong_password("Abcd 123@") == False
```

```
print("All password validation test cases passed!")
```

Explanation:

The `is_strong_password()` function checks whether a password has at least 8 characters and contains no spaces. It verifies the presence of uppercase letters, lowercase letters, digits, and special characters. The assert statements are used to test the password validation with different test cases. If the password meets all the requirements, the function returns True; otherwise, it returns False. If all test cases pass successfully, the program displays "All password validation test cases passed!"

Output:

A screenshot of a code editor's output window. The window has tabs for 'Problems', 'Output', 'Debug Console', and 'Code'. The 'Output' tab is active, showing the command `python -u "c:\Users\adith\OneDrive\Desktop\8.1\task-8.1.py"` and the output `All password validation test cases passed!`. Below this, it says `[Done] exited with code=0 in 0.234 seconds`.

```
Problems Output Debug Console ... Filter (e.g. text, !excludeText, t... Code
[Running] python -u "c:\Users\adith\OneDrive\Desktop\8.1\task-8.1.py"
All password validation test cases passed!

[Done] exited with code=0 in 0.234 seconds
```

Task Description #2 (Number Classification with Loops – Apply AI for Edge Case Handling)

Task: Use AI to generate at least 3 assert test cases for a `classify_number(n)` function. Implement using loops.

Input:

```
def classify_number(n):
    if n is None or isinstance(n, str):
        return "Invalid Input"

    if n > 0:
        return "Positive"

    elif n < 0:
```

```
        return "Negative"

    else:

        return "Zero"


# Assert test cases

assert classify_number(10) == "Positive"
assert classify_number(-5) == "Negative"
assert classify_number(0) == "Zero"
assert classify_number(1) == "Positive"
assert classify_number(-1) == "Negative"
assert classify_number("10") == "Invalid Input"
assert classify_number(None) == "Invalid Input"


print("Classification logic passing all assert tests.")
```

Explanation:

The `classify_number()` function classifies a number as Positive, Negative, or Zero and handles invalid inputs such as strings and None. The assert statements test normal values and boundary cases like -1, 0, and 1. If all tests pass, the program displays “Classification logic passing all assert tests.”

Output:

```
[Running] python -u "c:\Users\adith\OneDrive\Desktop\8.1\task-8.2.py"  
Classification logic passing all assert tests.  
  
[Done] exited with code=0 in 0.26 seconds
```

Task Description #3 (Anagram Checker – Apply AI for String Analysis)

Task: Use AI to generate at least 3 assert test cases for `is_anagram(str1, str2)` and implement the function.

Input:

```
def is_anagram(str1, str2):  
    # Ignore case, spaces, and punctuation  
    s1 = ''.join(char.lower() for char in str1 if char.isalnum())  
    s2 = ''.join(char.lower() for char in str2 if char.isalnum())  
  
    return sorted(s1) == sorted(s2)  
  
# Assert test cases  
assert is_anagram("listen", "silent") == True  
assert is_anagram("hello", "world") == False  
assert is_anagram("Dormitory", "Dirty Room") == True  
assert is_anagram("", "") == True  
assert is_anagram("The eyes!", "They see") == True
```

```
assert is_anagram("cat", "dog") == False
```

```
print("Anagram function passed all AI-generated tests.")
```

Explanation:

The `is_anagram()` function removes spaces and punctuation and converts both strings to lowercase before comparing them.

It uses `sorted()` to check whether both strings contain the same characters, including edge cases like empty or identical strings.

The assert statements verify different anagram and non-anagram cases, and all tests pass successfully.

Output:

```
[Running] python -u "c:\Users\adith\OneDrive\Desktop\8.1\task-8.3.py"
Anagram function passed all AI-generated tests.

[Done] exited with code=0 in 0.25 seconds
```

Task Description #4 (Inventory Class – Apply AI to Simulate Real-World Inventory System)

🔗 Task: Ask AI to generate at least 3 assert-based tests for an Inventory class with stock management.

Input:

```
class Inventory:

    def __init__(self):

        self.stock = {}
```

```
def add_item(self, name, quantity):  
    self.stock[name] = self.stock.get(name, 0) + quantity  
  
def remove_item(self, name, quantity):  
    if name not in self.stock or self.stock[name] < quantity:  
        return False  
    self.stock[name] -= quantity  
    return True  
  
def get_stock(self, name):  
    return self.stock.get(name, 0)
```

Assert-based test cases

```
inv = Inventory()
```

```
inv.add_item("Pen", 10)  
assert inv.get_stock("Pen") == 10
```

```
inv.remove_item("Pen", 5)  
assert inv.get_stock("Pen") == 5
```

```
inv.add_item("Book", 3)  
assert inv.get_stock("Book") == 3
```

```
print("Inventory class passed all assertions.")
```

Explanation:

The Inventory class manages items and their quantities using a dictionary.

The `add_item()`, `remove_item()`, and `get_stock()` methods handle adding, removing, and checking stock.

The assert statements test the inventory operations, and all tests pass successfully.

Output:

```
[Running] python -u "c:\Users\adith\OneDrive\Desktop\8.1\task-8.4.py"
Inventory class passed all assertions.

[Done] exited with code=0 in 0.251 seconds
```

Task Description #5 (Date Validation & Formatting – Apply AI for Data Validation)

🔗 Task: Use AI to generate at least 3 assert test cases for `validate_and_format_date(date_str)` to check and convert dates.

Input:

```
from datetime import datetime
```

```
def validate_and_format_date(date_str):
    try:
        date = datetime.strptime(date_str, "%m/%d/%Y")
        return date.strftime("%Y-%m-%d")
    except ValueError:
        return "Invalid Date"
```

```
# Assert test cases

assert validate_and_format_date("10/15/2023") == "2023-10-15"

assert validate_and_format_date("02/30/2023") == "Invalid Date"

assert validate_and_format_date("01/01/2024") == "2024-01-01"

assert validate_and_format_date("02/29/2024") == "2024-02-29"

assert validate_and_format_date("13/10/2023") == "Invalid Date"


print("Date validation passed all AI-generated assertions.")
```

Explanation:

The function validates dates in MM/DD/YYYY format using Python's datetime module. It converts valid dates into YYYY-MM-DD format and returns "Invalid Date" for incorrect dates.

The assert statements test valid dates, invalid dates, and leap-year edge cases to ensure the function works correctly.

Output:

```
[Running] python -u "c:\Users\adith\OneDrive\Desktop\8.1\task-8.5.py"
Date validation passed all AI-generated assertions.

[Done] exited with code=0 in 0.253 seconds
```